Agentic Memory Atomicity

A Transactional Memory Architecture
for Ephemeral AI Agent Systems

Loïc Mancino & Jérôme Chincarini

Synergix Labs, a division of Synergix SA

Project: **WRAI.TH**

`https://github.com/Synergix-lab/WRAI.TH`

March 2026

Based on one year of R&D at Synergix Labs

AGENTIC MEMORY ATOMICITY: A TRANSACTIONAL MEMORY ARCHITECTURE FOR EPHEMERAL AI AGENT SYSTEMS

SYNERGIX LABS – WRAI.TH

Loïc Mancino
Synergix Labs, a division of Synergix SA
loic@synergixlabs.com

Jérôme Chincarini
Synergix Labs, a division of Synergix SA
jerome@synergixlabs.com

ABSTRACT

We present *Agentic Memory Atomicity*, an architecture where AI agents are stateless ephemeral processes that read from and commit to a persistent, transactional memory store. Each agent spawns, loads inherited knowledge, performs work, commits new knowledge atomically, and terminates. The next spawn inherits everything. We formalize the state model, demonstrate monotonic knowledge growth under compaction, and show sub-linear cost scaling for multi-agent teams sharing project-scoped memory. Our implementation (relay OS) uses SQLite with FTS5, MCP protocol, and Go, with mechanisms for conflict resolution, budget pruning, temporal garbage collection, and priority-based context injection. Simulations show convergence of marginal learning, decreasing cost per unit of work, and robustness to LLM noise. This work is the result of one year of R&D at Synergix Labs, a division of Synergix SA. The project is open source at `https://github.com/Synergix-lab/WRAI.TH`.

Keywords: multi-agent systems, persistent memory, transactional state, ephemeral compute, LLM agents

1 Introduction

Current LLM-based agent systems face a fundamental tension: agents need persistent knowledge to be effective, but long-running processes suffer from context window exhaustion, idle cost, and crash fragility.

We propose **Agentic Memory Atomicity** — a design where:

1. Agents are **ephemeral**: they spawn, work, and die.
2. Knowledge lives in a **persistent, transactional store** outside the agent.
3. Each spawn **inherits all prior knowledge** via a projection function.
4. Memory commits are **atomic**: success writes ΔM, crash preserves M_t.

Contributions. We make the following contributions:

- A formal state model for ephemeral agents with three memory scopes, three layers, and cascade lookup.
- The *Ephemeral Agent Principle*: agents carry no internal state between invocations; intelligence resides in the persistent memory store.
- A transactional memory commit protocol with conflict detection, versioning, and budget pruning.
- Temporal garbage collection mechanisms: message TTL, delivery state machines, ACK escalation, agent staleness detection, and ghost reaping.
- A utility scoring formula for priority-based context injection with P0 bypass.
- Simulations demonstrating convergence, sub-linear multi-agent cost, and robustness to LLM noise.
- A production implementation (relay OS) in Go with SQLite+FTS5 and MCP protocol, open source at `https://github.com/Synergix-lab/WRAI.TH`.

Table 1: Comparison of multi-agent memory architectures.

System	Memory Model	Agent Lifecycle	Limitation
LangGraph	Graph state (in-memory)	Persistent process	Context exhaustion
CrewAI	Shared task queue	Persistent threads	No transactional memory
AutoGen	Conversation history	Session-bound	Lost on termination
Relay OS (ours)	Transactional DB	Ephemeral spawns	—

2 Background & Related Work

Existing frameworks treat memory as a side-effect of agent execution. We treat it as the **primary artifact** — agents are disposable compute, memory is the durable intelligence:

$$\text{Intelligence} = f(M_t), \quad \text{Agent} = \text{ephemeral compute over } M_t$$

3 Agent State Model

Definition 1 (Agent State). The state of an agent at spawn time t is:

$$S_t(A) = \{I, M_t, V, T_t, \Omega_t, R\}$$

where I is identity (profile, role, reports_to), M_t is memories at time t, V is vault (conventions, docs), T_t is assigned tasks, Ω_t is inbox (unread messages, TTL-bound), and R is rules (boot, cycle, learning).

Definition 2 (Memory Scopes). Memory is partitioned into three scopes with cascade lookup:

$$M_t = M_t^{\text{agent}} \cup M_t^{\text{project}} \cup M_t^{\text{global}}$$

Table 2: Memory scope visibility and cascade priority.

Scope	Visibility	Priority
`agent`	Private to owner	1 (checked first)
`project`	All agents in project	2
`global`	All agents everywhere	3 (fallback)

Definition 3 (Memory Layers). Each memory has a layer that determines override precedence:

$$\texttt{constraints} > \texttt{behavior} > \texttt{context}$$

A `behavior` write cannot override a `constraints` memory. A CTO can set constraints that workers cannot bypass.

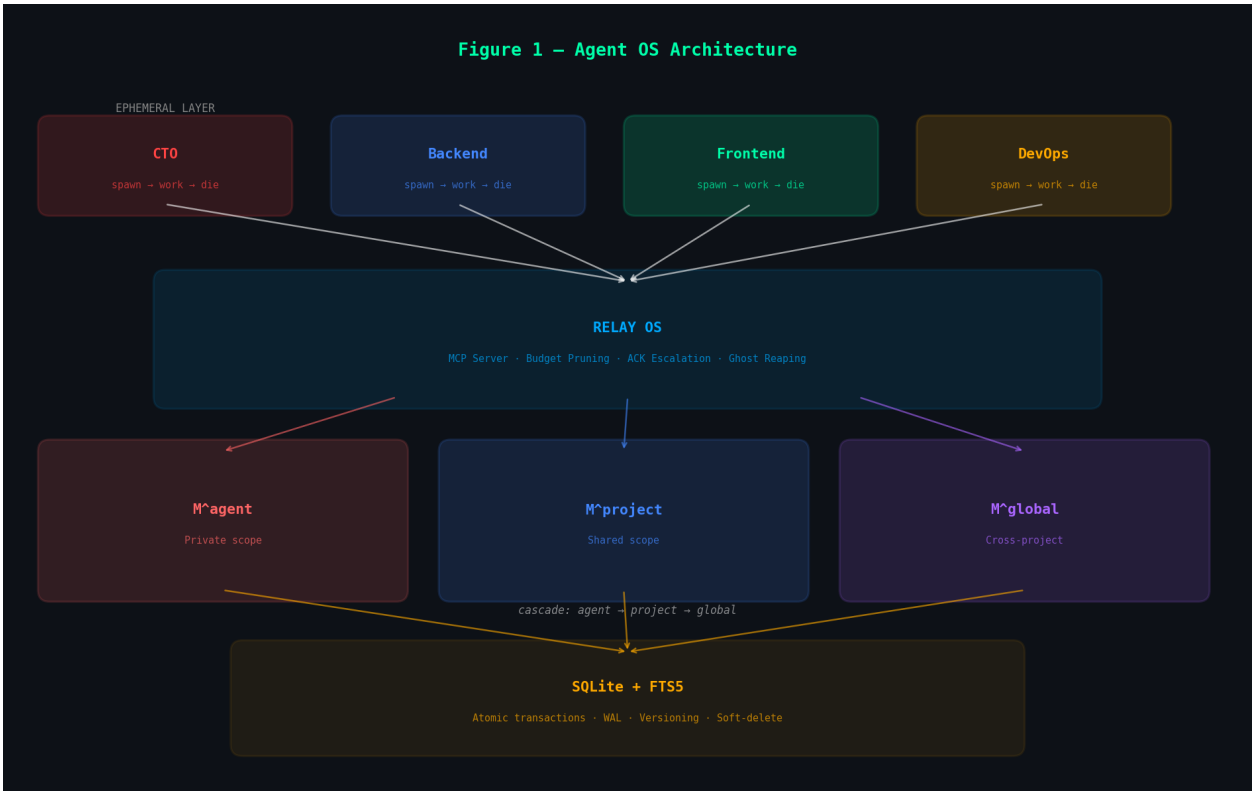

Figure 1: Agent OS architecture: ephemeral agents communicate through the relay, which manages three memory scopes persisted in SQLite with FTS5.

4 Ephemeral Agent Principle

Agents are **stateless compute processes**. They carry no internal state between invocations:

$$\text{Agent} = f(M_t) \rightarrow (W, \Delta M)$$

The intelligence of the system resides entirely in M_t. An agent is a pure function that: (1) spawns, (2) registers with the relay, (3) loads M_t via `BuildSpawnContext()`, (4) computes work W, (5) commits ΔM, and (6) terminates.

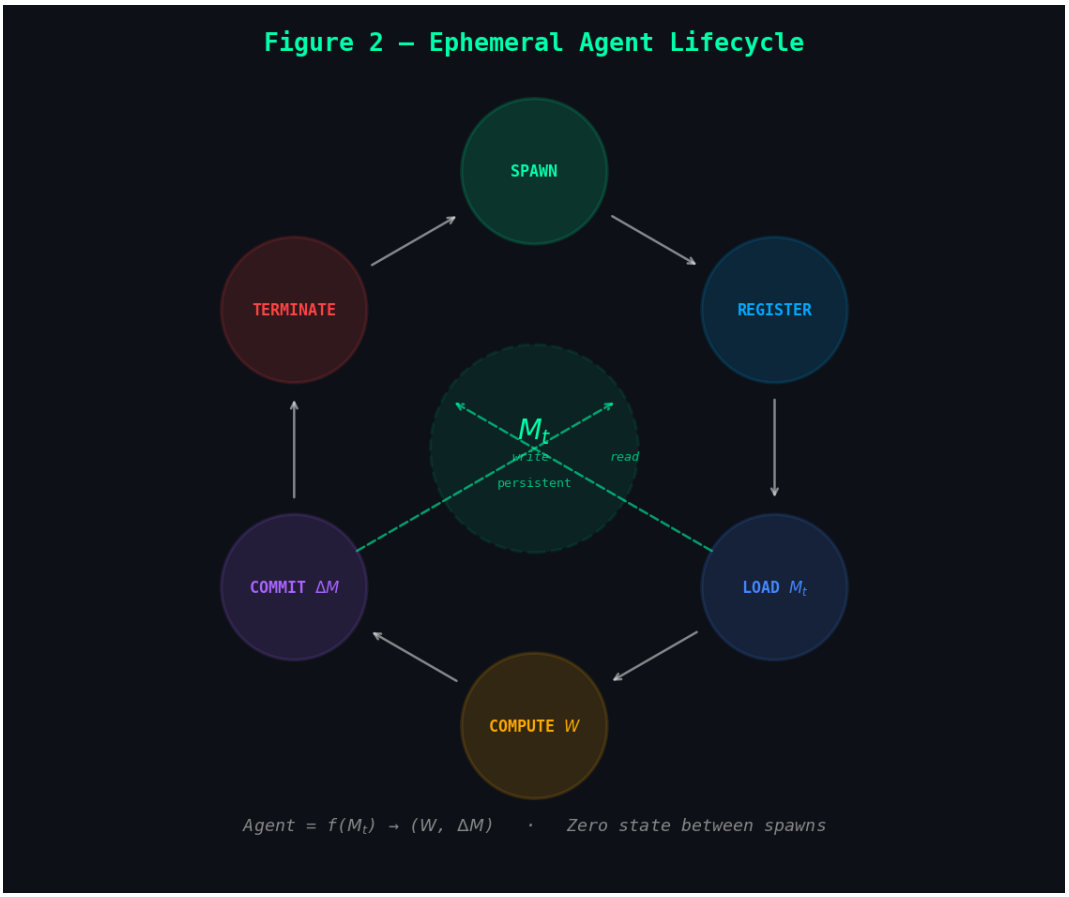

Figure 2: Ephemeral agent lifecycle: spawn → register → load M_t → compute W → commit ΔM → terminate. The persistent memory store M_t is the only durable artifact.

Table 3: Persistent vs. ephemeral agent comparison.

Property	**Persistent Agent**	**Ephemeral Agent**
Context window	Exhausts → loses context	Always fresh
Crash recovery	Corrupted state possible	M_t intact
Idle cost	Tokens burned (keepalive)	Zero
Scalability	1 process = 1 agent	Pool of N spawns
Knowledge	Limited to session	Cumulative ∞
Formalism	$S_t = f(S_{t-1}, \text{input})$	$c : S_t \to (S_{t+1}, W)$

5 Atomic Memory Commit

Definition 4 (Cycle Transition). A cycle is a state transition:

$$c : S_t \to (S_{t+1}, W)$$

where W is work produced. The transition decomposes: $M_{t+1} = \text{compact}(M_t \cup \Delta M_{\text{valid}})$, $\Omega'_{t+1} = \Omega_t \cup \Delta\Omega$, and T_{t+1} reflects task updates.

Definition 5 (Atomicity).

$$M_{t+1} = \begin{cases} \text{compact}(M_t \cup \Delta M_{\text{valid}}) & \text{if cycle succeeds (exit 0)} \\ M_t & \text{if cycle crashes} \end{cases}$$

No intermediate corrupted state. Transaction commit or rollback.

Definition 6 (Validation Filter). Every memory write passes through:

$$\Delta M_{\text{valid}} = \text{filter}(\Delta M, M_t)$$

The filter enforces: (a) conflict detection — two agents writing same key with different values are both preserved until explicit resolution; (b) layer hierarchy — `behavior` cannot override `constraints`; (c) scope isolation; (d) deduplication — same key+value triggers timestamp update only.

Definition 7 (Compaction). Raw memory grows monotonically, but the projected view at boot is bounded:

$$|M_{\text{boot}}| \leq B_{\max}$$

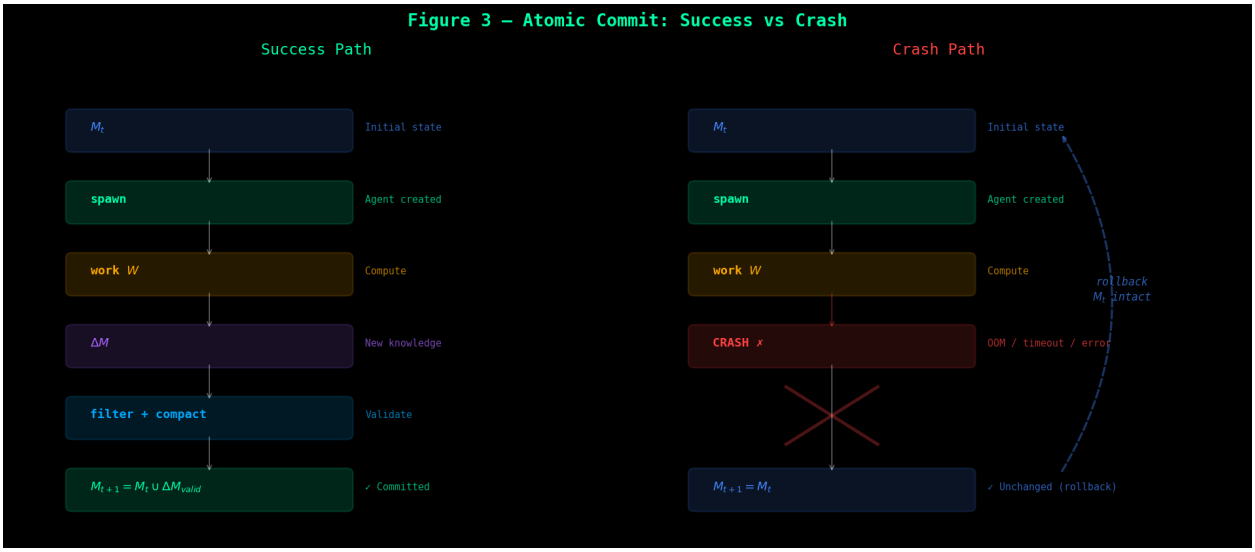

Figure 3: Atomic commit: success path commits $M_t \cup \Delta M_{\text{valid}}$; crash path preserves M_t unchanged (rollback).

6 Knowledge Evolution

Spawn Chain Inheritance. For two successive spawns s_1, s_2 of the same profile:

$$M(s_2) \supseteq M(s_1)$$

Agent s_2 knows everything s_1 learned. This is boot-time inheritance via `BuildSpawnContext()`, not shared runtime memory.

Inter-Agent Memory Elimination. An agent can supersede another's knowledge via `set_memory(key,` $v_{\texttt{new}}$`, scope:project, upsert:true)`. The old version is archived with a `supersedes` pointer; version is incremented. Complete traceability, never hard-delete:

$$v_1 \xrightarrow{\text{supersedes}} v_2 \xrightarrow{\text{supersedes}} v_3$$

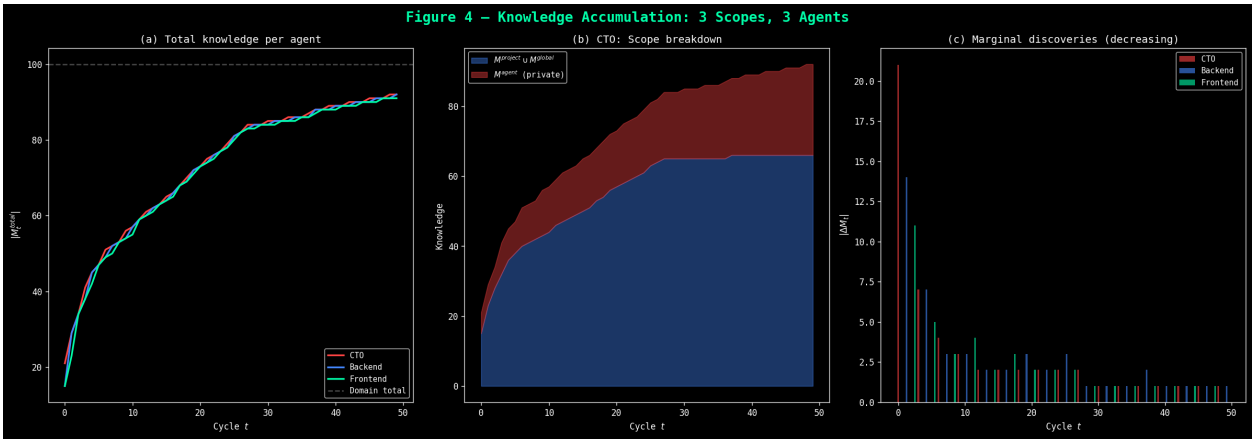

Figure 4: Knowledge accumulation with 3 agents and 3 memory scopes. (a) Total knowledge per agent converges. (b) CTO scope breakdown: shared vs. private. (c) Marginal discoveries $|\Delta M_t|$ decrease over cycles.

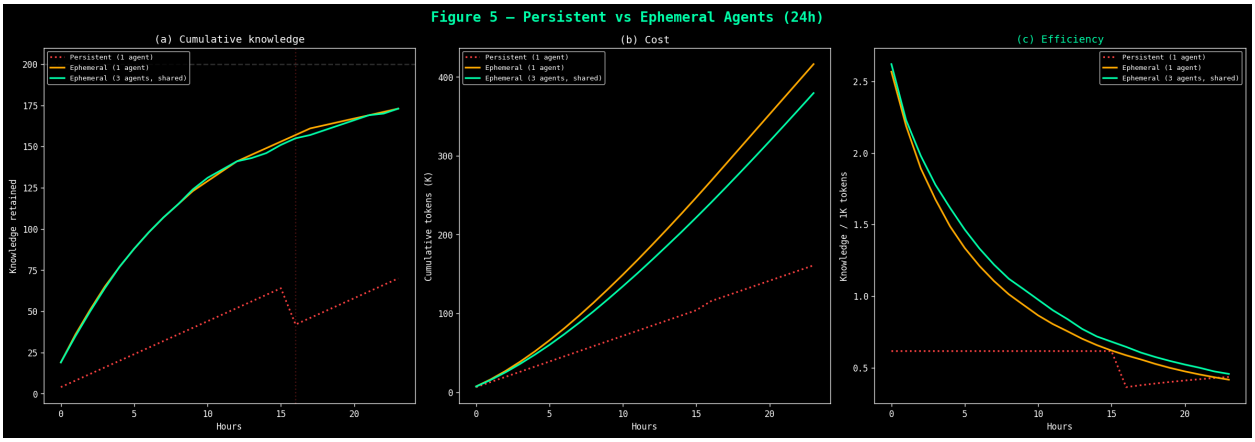

Figure 5: 24-hour comparison: persistent agent (context resets marked with dotted lines) vs. ephemeral agents. (a) Cumulative knowledge. (b) Token cost. (c) Efficiency (knowledge per 1K tokens). Ephemeral agents with shared project memory dominate on all three metrics.

7 Convergence Conjecture

Conjecture (Knowledge Saturation Hypothesis).

$$\lim_{n\to\infty} |\Delta M_{\text{valid},n}| \to 0 \quad \text{under finite domain and active filter}$$

This is **not a theorem**. It holds under three conditions: (1) the knowledge domain is finite; (2) the filter eliminates duplicates and contradictions; (3) the agent learns effectively (no hallucination loops). In practice, convergence is due to filter + compaction, not intrinsic LLM learning.

Corollary (Marginal Cost Decrease).

$$C_t = \underbrace{C_{\text{boot}}}_{\text{fixed}} + \underbrace{C_{\text{context}}(|M_t^{\text{vis}}|)}_{\text{bounded by } B_{\max}} + \underbrace{C_{\text{work}}}_{\text{constant}} + \underbrace{C_{\text{learn}}(|\Delta M_t|)}_{\text{decreasing}}$$

As $|\Delta M_t| \to 0$, cost converges to $C_{\text{boot}} + C_{\text{context}} + C_{\text{work}}$.

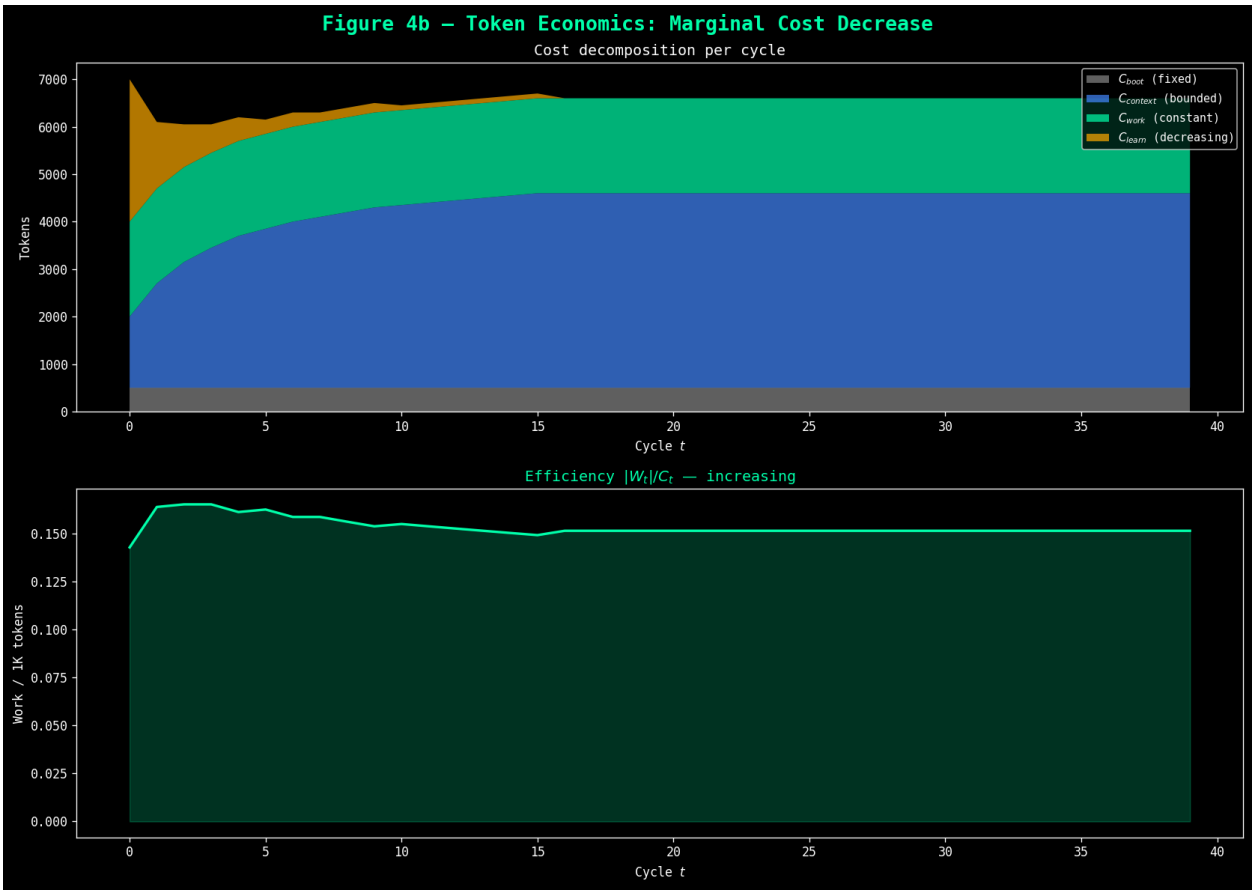

Figure 6: Token economics over 40 cycles. (Top) Cost decomposition: C_{learn} (orange) decreases to zero as knowledge saturates, while C_{context} (blue) is bounded by $B_{\max}$. (Bottom) Efficiency $|W|/C_t$ is monotonically increasing.

8 Memory Validation & Conflict Resolution

8.1 Validation Pipeline

Every memory write passes through a multi-stage pipeline:

$$\Delta M \xrightarrow{\text{dedup}} \xrightarrow{\text{layer check}} \xrightarrow{\text{scope isolation}} \xrightarrow{\text{conflict detect}} \xrightarrow{\text{commit}} M_{t+1}$$

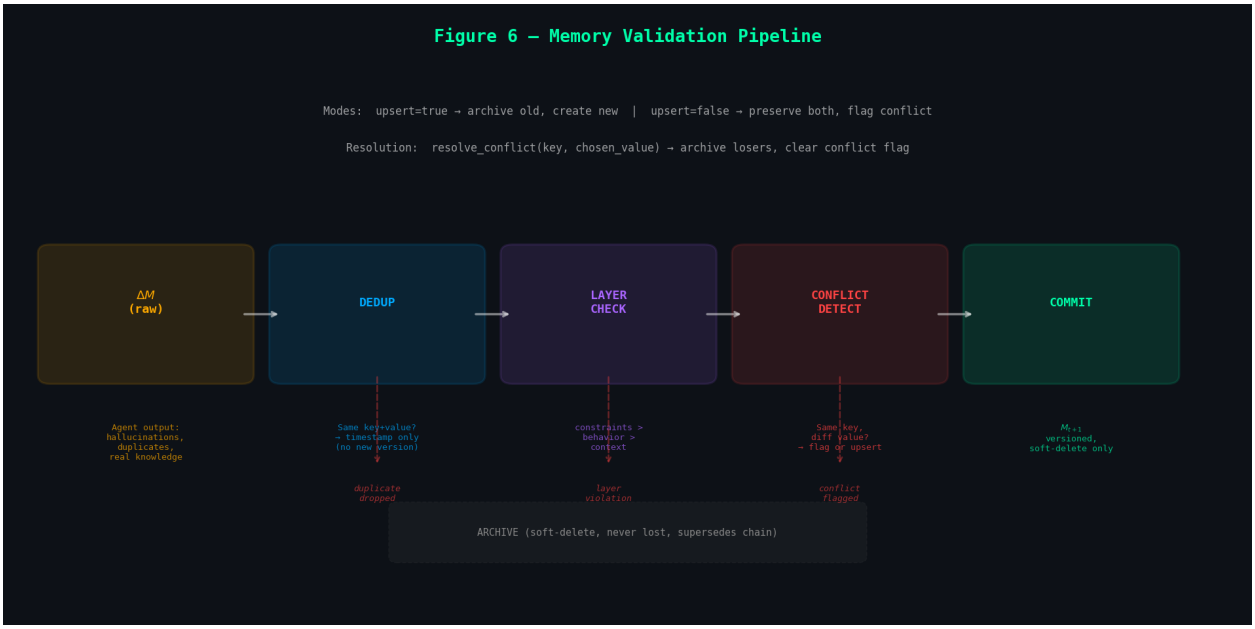

Figure 7: Memory validation pipeline: raw ΔM passes through deduplication, layer check, conflict detection, and commit. Rejections are archived, never lost.

8.2 Write Modes

Two modes are supported: (a) **upsert** (default) — archives old version via `supersedes` chain, creates new version silently; (b) **conflict detection** (`upsert=false`) — preserves both versions with `conflict_with` cross-references until explicit resolution via `resolve_conflict()`.

8.3 Budget Pruning

The relay applies a utility scoring function to select which messages fit within the agent's byte budget $B_{\max}$:

$$U(m) = 0.7 \times \text{priority}(m) + 0.2 \times \text{Jaccard}(\text{tags}_m, \text{tags}_{\text{agent}}) + 0.1 \times \text{freshness}(m)$$

where priority : $\text{P0} = 1.0, \text{P1} \approx 0.67, \text{P2} \approx 0.33, \text{P3} = 0$, Jaccard similarity measures tag overlap, and $\text{freshness}(m) = 1/(1 + \text{age}(m)/3600)$ provides exponential decay over one hour.

P0 (interrupt) always bypasses the budget. Remaining messages are scored and selected greedily until $B_{\max}$ is reached.

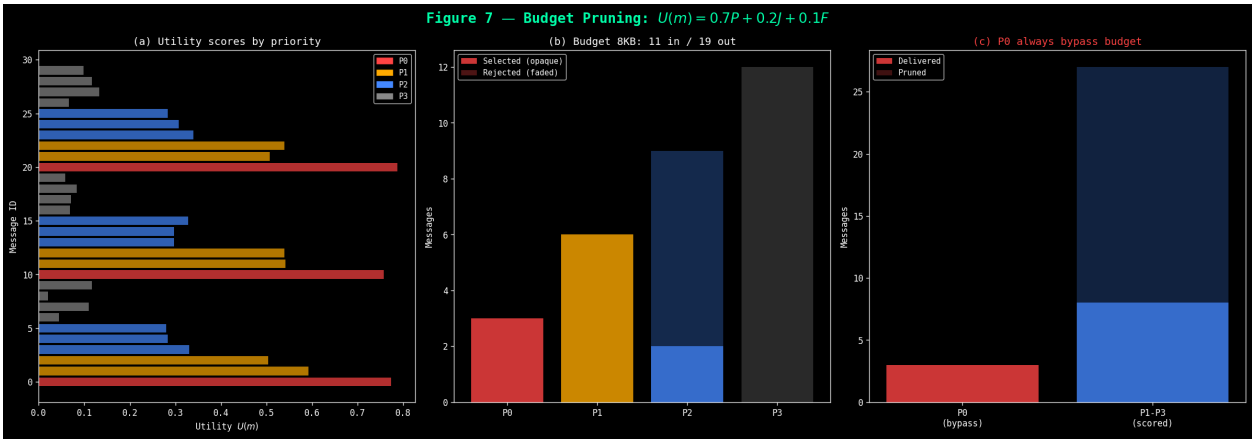

Figure 8: Budget pruning in action. (a) Utility scores by priority: P0 (red) dominates. (b) Budget allocation: 8KB selects high-utility messages, rejects low-priority ones. (c) P0 messages always bypass the budget — 100% delivery rate.

9 Implementation

The relay OS is implemented in Go and has been in production for one year at Synergix Labs.

Table 4: Implementation stack.

Component	Technology	Purpose
Relay server	Go	MCP server, HTTP API, SSE streaming
Storage	SQLite + FTS5	Transactional memory, full-text search
Agent runtime	Claude Code (headless)	LLM compute via MCP
Scheduling	Cron expressions	Agent lifecycle management
Protocol	MCP	Tool discovery, push notifications

Table 5: Key functions in the relay OS codebase.

Function	File	Role
`BuildSpawnContext()`	`spawn/prompt.go`	Projection: $M_t \to S_t$
`SetMemory()`	`db/memories.go`	Commit: $\Delta M \to M_{t+1}$
`ResolveConflict()`	`db/memories.go`	Conflict resolution
`applyBudget()`	`relay/budget.go`	Utility scoring
`StartCleanup()`	`relay/cleanup.go`	Temporal GC
`FormatPrompt()`	`spawn/prompt.go`	$S_t \to$ prompt string

9.1 Temporal Mechanisms

The relay manages temporality at six levels:

10 Experiments

We present three simulation experiments validating the core claims.

10.1 Experiment 1: Threats Without Protection

We simulate four threats in the absence of protection mechanisms: (1) LLM noise causing memory bloat with quality degradation; (2) boot cost explosion without $B_{\max}$ bound; (3) conflict probability scaling with N agents; (4) inbox pollution without message TTL.

Table 6: Temporal garbage collection mechanisms.

Mechanism	**Interval**	**Timeout**
Message TTL	On send	Default 4h, 0 = immortal
Delivery state machine	queued → surfaced → ack → expired	Cascades from message TTL
ACK escalation	Every 5min	15min → notify, 45min → escalate
Agent staleness	Every 5min	30min inactivity → inactive
File lock TTL	Every 5min	Default 30min
Ghost reaping	Every 30s	Zombie spawns → dead

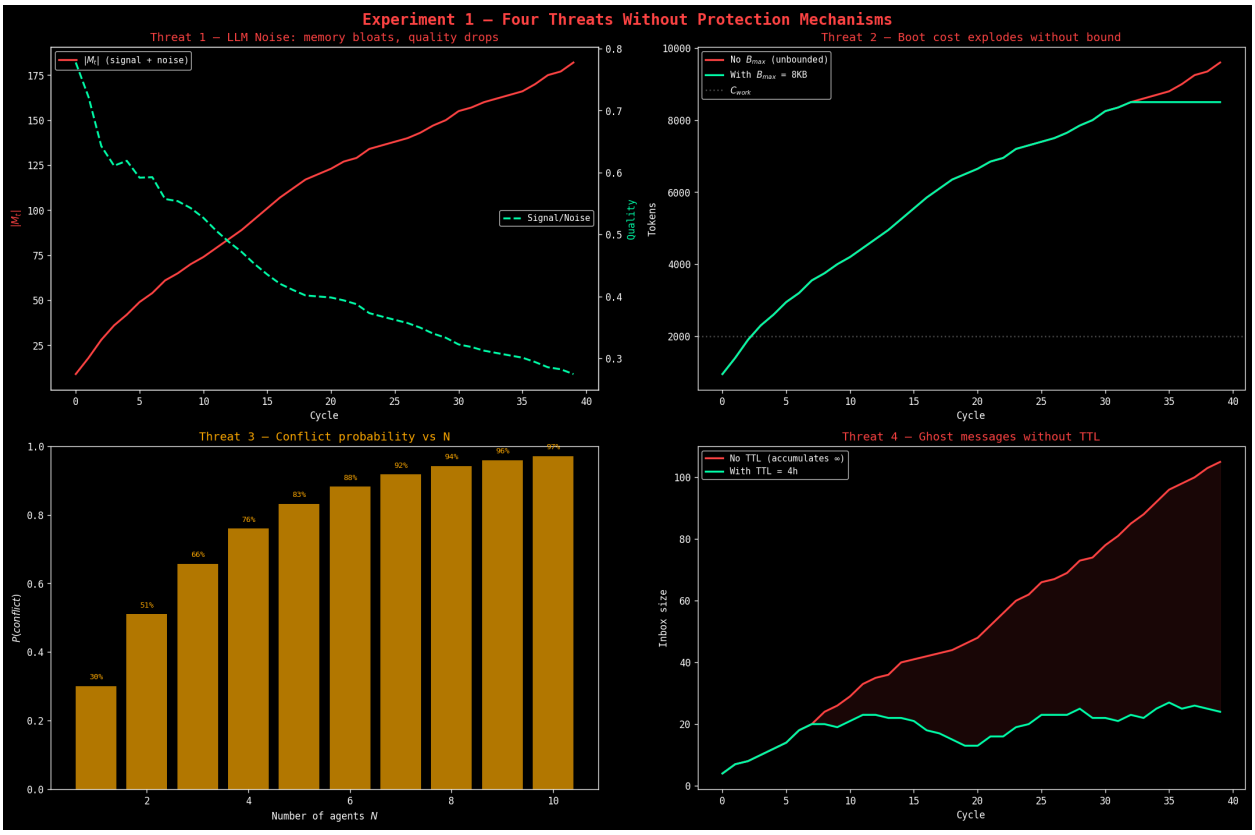

Figure 9: Four threats without protection mechanisms. Top-left: LLM noise inflates $|M_t|$ while signal/noise ratio drops. Top-right: boot cost grows unboundedly vs. bounded by $B_{\max}$. Bottom-left: conflict probability $P(\text{conflict}) = 1 - (1 - 0.3)^N$ reaches 97% at $N = 10$. Bottom-right: inbox size grows infinitely without TTL.

10.2 Experiment 2: Temporal Mechanisms

We simulate the relay's temporal mechanisms over a 2-hour window, showing message TTL enforcement, agent staleness detection, ACK escalation timeline, and file lock auto-release.

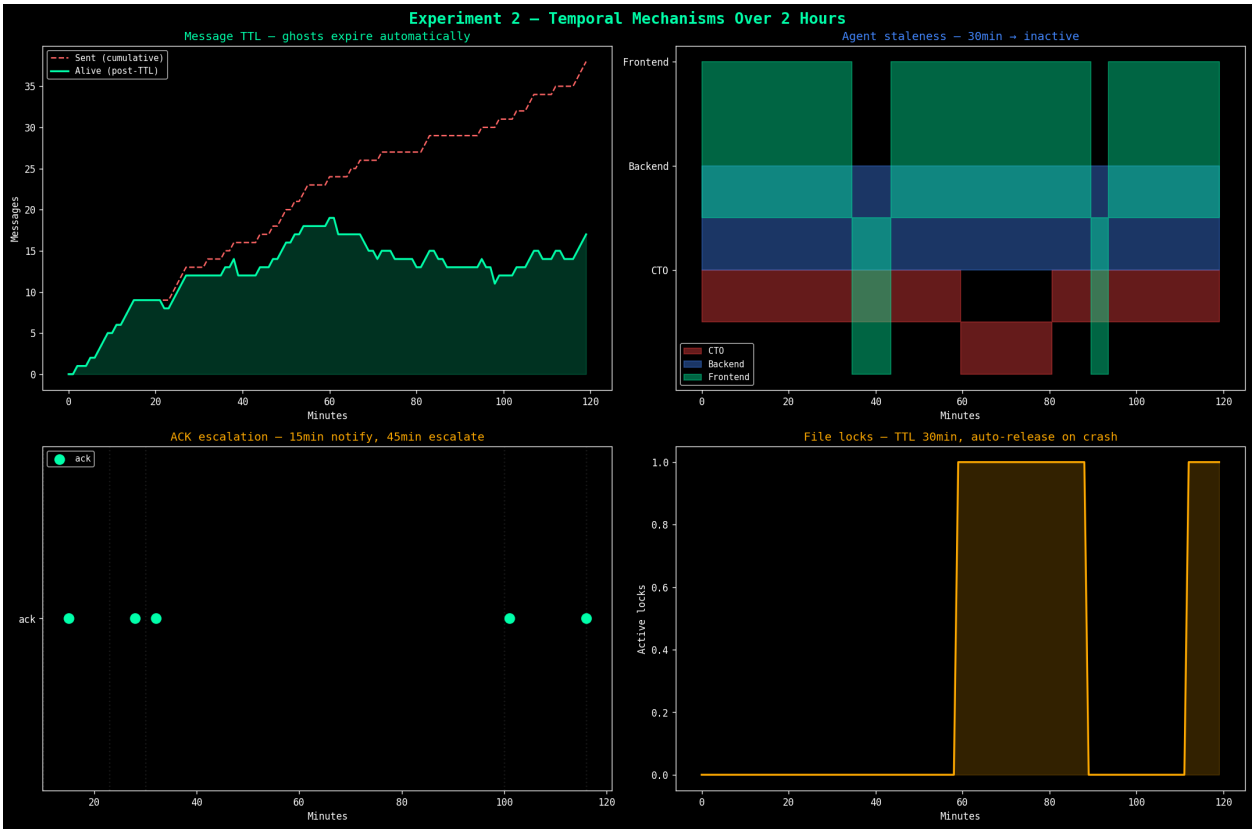

Figure 10: Temporal mechanisms over 2 hours. Top-left: messages expire via TTL, keeping inbox bounded. Top-right: agent staleness detection after 30min. Bottom-left: ACK escalation at 15min (notify) and 45min (escalate). Bottom-right: file locks auto-release after 30min TTL.

10.3 Experiment 3: Multi-Agent Conflict & Resolution

We simulate 4 agents writing to 10 shared keys over 40 cycles, with 40% conflict rate and CTO-mediated resolution.

Figure 11: Multi-agent conflict detection and resolution. (a) Active conflicts remain bounded as CTO resolves them. (b) Event timeline showing creates, updates, supersedes, conflicts, and resolutions per agent. (c) Version growth via supersedes chains.

11 Discussion & Conclusion

11.1 Axioms

Table 7: Axioms of Agentic Memory Atomicity.

#	Axiom	Statement
A1	Monotonicity	$M_{t+1} \supseteq M_t$ modulo compaction
A2	Atomicity	Cycle produces $(\Delta M, \Delta\Omega, W)$ or nothing
A3	Inheritance	$M(s_{n+1}) \supseteq M(s_n)$ (subject to $B_{\max}$)
A4	Saturation	$\lim \lvert\Delta M_n\rvert \to 0$ (conjecture)
A5	Zero idle	Terminated agent = 0 cost
A6	Scopes	$M = M^a \cup M^p \cup M^g$, cascade lookup
A7	Layers	constraints > behavior > context

11.2 Implemented Mechanisms

Table 8: Protection mechanisms and the threats they address.

#	Mechanism	Addresses
R1	Budget pruning ($U = 0.7P + 0.2J + 0.1F$)	LLM noise, boot cost
R2	Temporal GC (TTL, staleness, ACK, reaping)	Stale data, zombies
R3	Conflict resolution (upsert/conflict modes)	Multi-agent divergence
R4	Tool security ($\text{tools}_{\text{eff}} = \text{tools}_c \cap \text{tools}_p$)	Privilege escalation
R5	Versioning (supersedes, soft-delete)	Provenance, audit

11.3 Key Results

From our axioms and mechanisms:

- **C1**: A1 + A3 + R3 ⇒ knowledge is durable and shared (3 scopes, conflicts resolved, elimination traceable).
- **C2**: A2 + A5 + R2 ⇒ cost is proportional to work (no idle waste, auto-cleanup).
- **C3**: A4 + R1 ⇒ efficiency is empirically increasing ($C_{\text{learn}} \searrow$, C_{boot} bounded).
- **C4**: A6 + R3 ⇒ multi-agent cost is sub-linear: $C(N) < N \times C_{\text{solo}}$.

11.4 Limitations

- Convergence (A4) is empirical, not proven — LLM noise can prevent it.
- Budget pruning may discard relevant low-priority knowledge under extreme pressure.

11.5 Future Work

- Formal convergence proof under bounded noise assumptions.
- Cross-relay federation for multi-organization agent networks.

The agent does not live. It spawns, works, learns, dies. The next one knows everything.